

Nazwa przedmiotu	Administracja systemów rozproszonych
Prowadzący zajęcia	mgr inż. Dariusz Rączka
Tryb zajęć	Projekt
Imię i nazwisko	Adrian Wrona
Grupa laboratoryjna	L-6
Temat projektu	Wdrożenie środowiska rozproszonego w chmurze Azure z użyciem Infrastructure as Code



Spis treści

Wstęp	3
Przygotowanie środowiska pracy	4
Projektowanie infrastruktury bazowej (Sieć i Monitoring)	6
Wdrażanie usług aplikacyjnych i bazodanowych	8
Próba zautomatyzowania zarządzania użytkownikami (Microsoft Entra ID)	10
Wdrożenie aplikacji kontenerowej i konfiguracja dostępu sieciowego	12
Podsumowanie	14



Wstęp

Celem projektu było zaprojektowanie oraz w pełni zautomatyzowane wdrożenie rozproszonego środowiska chmurowego w usłudze Microsoft Azure. Zamiast manualnego konfigurowania zasobów poprzez portal przeglądarkowy, wykorzystano podejście Infrastructure as Code (IaC) przy pomocy narzędzia Terraform. Zadanie polegało na zdefiniowaniu w postaci kodu (plik main.tf) kompletnej infrastruktury składającej się z odizolowanej sieci wirtualnej (VNet), relacyjnej bazy danych (PostgreSQL), systemu monitorowania (Log Analytics) oraz środowiska uruchomieniowego dla kontenerów (Azure Container Apps). Projekt miał na celu wykazanie powtarzalności wdrożeń oraz umiejętności radzenia sobie z ograniczeniami nakładanymi na subskrypcje edukacyjne. W trakcie projektu używałem VS code, który miał pobrane środowisko Terraform. Zainstalowałem do tego również AzureCLI z oficjalnej strony Microsoft. Stworzyłem folder do pracy i po zalogowaniu się do Azure w przeglądarce (konto studenckie) i w VS mogłem przystąpić do dalszej pracy.

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. Wszelkie prawa zastrzeżone.

C:\Users\adria\Terraform>cd .\wahlnetwork\tf-examples-aws\azure\
System nie może odnaleźć określonej ścieżki.

C:\Users\adria\Terraform>cd
C:\Users\adria\Terraform
C:\Users\adria\Terraform>
* History restored

Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Wszelkie prawa zastrzeżone.

C:\Users\adria\Terraform>az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Azure for Students  a8e6dff8-d6a3-448e-9254-ed7b992e75f0  ICBPANPCSS-8009567-Politechnika Rzeszowska im. Ignacego Łukasiewicza

The default is marked with an *; the default tenant is 'ICBPANPCSS-8009567-Politechnika Rzeszowska im. Ignacego Łukasiewicza' and subscription is 'Azure for Students'
(a8e6dff8-d6a3-448e-9254-ed7b992e75f0).

Select a subscription and tenant (Type a number or Enter for no changes): 1

Tenant: ICBPANPCSS-8009567-Politechnika Rzeszowska im. Ignacego Łukasiewicza
Subscription: Azure for Students (a8e6dff8-d6a3-448e-9254-ed7b992e75f0)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.

C:\Users\adria\Terraform>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.100.0"...
- Installing hashicorp/azurerm v3.100.0...
- Installed hashicorp/azurerm v3.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
```



Przygotowanie środowiska pracy

Skupiłem się na przygotowaniu i zainicjowaniu środowiska uruchomieniowego dla narzędzia Terraform. Realizację zadania rozpocząłem od utworzenia głównego pliku konfiguracyjnego, w którym zdefiniowałem blok terraform. Wskazałem w nim wymagane zależności, deklarując wykorzystanie oficjalnego dostawcy azurem w stabilnej wersji 3.100.0. Krok ten był kluczowy, aby zapewnić pełną kompatybilność kodu z interfejsem programistycznym (API) chmury Microsoft Azure:

```
# 1. Blok konfiguracji Terraforma i określenie "providera"
terraform {
  required_providers {
    azurem = {
      source  = "hashicorp/azurem"
      version = "~> 3.100.0" # Używamy stabilnej wersji wtyczki dla Azure
    }
  }
}
```

Następnie przeprowadziłem właściwą inicjalizację dostawcy poprzez utworzenie bloku provider. Wewnątrz tego bloku skonfigurowałem specyficzne zachowania narzędzia, wprowadzając istotną modyfikację w sekcji features. Poprzez nadpisanie flagi `prevent_deletion_if_contains_resources` na wartość `false`, celowo wyłączyłem domyślne zabezpieczenie blokujące usuwanie pełnych grup zasobów. Działanie to było niezbędnym zabiegiem pozwalającym Terraformowi na zautomatyzowane i bezkolizyjne czyszczenie osieroconej infrastruktury podczas moich późniejszych prób obejścia restrykcyjnych limitów regionalnych narzuconych na studencką subskrypcję:

```
# 2. Inicjalizacja providera Azure
provider "azurem" {
  features {
    resource_group {
      prevent_deletion_if_contains_resources = false
    }
  }
}
```

```
C:\Users\adria\Terraform>terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v3.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

W celu umożliwienia wdrażania usług kontenerowych (Azure Container Apps), konieczne było uprzednie przygotowanie subskrypcji. Za pomocą interfejsu wiersza poleceń Azure CLI wywołałem polecenie `az provider register --namespace Microsoft.App`, rejestrując tym samym brakującego dostawcę zasobów na poziomie konta studenckiego. Następnie cyklicznie weryfikowałem status operacji komendą sprawdzającą, aż uległ on zmianie z "Registering" na "Registered", co trwale odblokowało możliwość tworzenia środowisk kontenerowych za pomocą narzędzia Terraform.

```
C:\Users\adria\Terraform>az provider register --namespace Microsoft.App
Registering is still on-going. You can monitor using 'az provider show -n Microsoft.App'

C:\Users\adria\Terraform>az provider show --namespace Microsoft.App --query "registrationState"
"Registering"

C:\Users\adria\Terraform>az provider show --namespace Microsoft.App --query "registrationState"
"Registered"
```

Projektowanie infrastruktury bazowej (Sieć i Monitoring)

Następnie przeszedłem do właściwego projektowania infrastruktury bazowej, która miała stanowić bezpieczny fundament pod docelowe usługi. Kontynuując pracę w edytorze Visual Studio Code w pliku main.tf, rozpocząłem od zdefiniowania głównego kontenera logicznego dla całego projektu, czyli Grupy Zasobów (Resource Group). Zadeklarowałem zasób `azurerm_resource_group`, nadając mu nazwę `rg-projekt-chmurowy_asr`. Jako lokalizację bazową wskazałem region `swedencentral` (Szwecja), co było przemyślaną decyzją wynikającą z wcześniejszej analizy limitów i restrykcyjnych polityk uczelnianych, które blokowały wdrożenia w innych popularnych lokalizacjach, choć na początku ustawiłem region na `eastusl` co było problemem w dalszym etapie projektu.

```
# 3. Grupa Zasobów (Resource Group)
resource "azurerm_resource_group" "rg" {
  name      = "rg-projekt-chmurowy_asr" # Nazwa widoczna w portalu Azure
  location  = "swedencentral"
}
```

Następnie zająłem się warstwą sieciową. Zdefiniowałem prywatną sieć wirtualną (VNet) o nazwie `vnet-projekt-chmurowy`, przypisując jej szeroką przestrzeń adresową `10.0.0.0/16`. Sieć tę powiązałem bezpośrednio z utworzoną wcześniej Grupą Zasobów, wykorzystując wbudowany w Terraforma mechanizm dynamicznych referencji (odwołując się do `azurerm_resource_group.rg.name`). Zapewniło to automatyczne budowanie relacji (zależności) przez narzędzie bez konieczności ręcznego definiowania kolejności tworzenia zasobów. Wewnątrz tej przestrzeni sieciowej wydzieliłem mniejszą, dedykowaną podsieć (Subnet) o nazwie `snet-aplikacja` z węższą pulą adresową `10.0.1.0/24`. Została ona zaprojektowana jako bezpieczne, odizolowane środowisko do wewnętrznej komunikacji przyszłych komponentów systemu.

```
# 4. Sieć wirtualna (VNet)
resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-projekt-chmurowy"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  address_space       = ["10.0.0.0/16"] # Pula adresów IP dla całej sieci
}

# 5. Podsieć (Subnet) dla aplikacji
resource "azurerm_subnet" "subnet" {
  name                = "snet-aplikacja"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"] # Pula adresów IP tylko dla tej podsieci
}
```

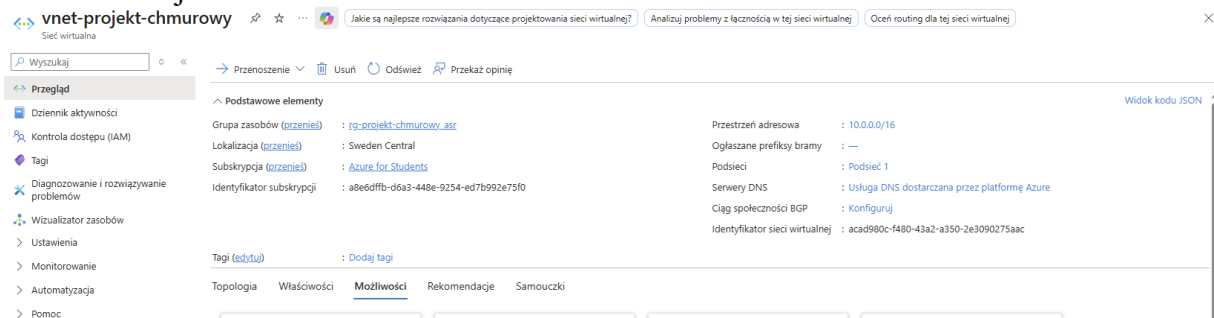
Ostatnim krokiem w tym etapie było wdrożenie scentralizowanego systemu monitoringu. W tym celu zadeklarowałem w kodzie usługę Azure Log Analytics Workspace, nazywając ją law-projekt-chmurowy. Aby zoptymalizować koszty w ramach subskrypcji studenckiej, wybrałem standardowy pakiet rozliczeniowy (SKU PerGB2018) i zdefiniowałem czas retencji (przechowywania) danych na 30 dni. Ten komponent, podobnie jak warstwa sieciowa, został spięty w kodzie ze szwedzkim regionem oraz główną Grupą Zasobów, co ostatecznie zamknęło proces budowy spójnego szkieletu infrastruktury, w pełni gotowego na przyjęcie bazy danych i środowiska kontenerowego. Cały nowo dopisany kod na bieżąco weryfikowałem i zapisywałem w środowisku Visual Studio Code, dbając o spójność i czytelność konfiguracji.

```
# 6. Monitoring (Log Analytics Workspace)
resource "azurerm_log_analytics_workspace" "law" {
  name                = "law-projekt-chmurowy"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  sku                 = "PerGB2018"
  retention_in_days   = 30 # Jak długo trzymać logi (30 dni)
}
```

```
azurerm_resource_group.rg: Creating...
azurerm_resource_group.rg: Creation complete after 7s [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy]
azurerm_virtual_network.vnet: Creating...
azurerm_log_analytics_workspace.law: Creating...
azurerm_virtual_network.vnet: Still creating... [00m10s elapsed]
azurerm_log_analytics_workspace.law: Still creating... [00m10s elapsed]
azurerm_virtual_network.vnet: Creation complete after 12s [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy]
azurerm_subnet.subnet: Creating...
azurerm_subnet.subnet: Creation complete after 6s [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy/subnets/snet-aplikacja]
azurerm_log_analytics_workspace.law: Still creating... [00m20s elapsed]
azurerm_log_analytics_workspace.law: Still creating... [00m30s elapsed]
azurerm_log_analytics_workspace.law: Still creating... [00m40s elapsed]
azurerm_log_analytics_workspace.law: Creation complete after 47s [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy/providers/Microsoft.OperationalInsights/workspaces/law-projekt-chmurowy]

Apply complete! Resources: 4 added, 0 changed, 1 destroyed.
```

Weryfikacja wdrożenia infrastruktury bazowej w Portalu Azure. Widok szczegółowy utworzonej sieci wirtualnej (VNet), potwierdzający poprawne zaaplikowanie konfiguracji zdefiniowanej w kodzie Terraform.



The screenshot shows the Azure Portal interface for a virtual network. The top navigation bar includes the search bar and several tabs: 'vnet-projekt-chmurowy', 'Jakie są najlepsze rozwiązania dotyczące projektowania sieci wirtualnej?', 'Analizuj problemy z łącznością w tej sieci wirtualnej', and 'Oceń routing dla tej sieci wirtualnej'. The left sidebar contains a list of navigation options: 'Przegląd', 'Dziennik aktywności', 'Kontrola dostępu (IAM)', 'Tagi', 'Diagnozowanie i rozwiązywanie problemów', 'Wizualizator zasobów', 'Ustawienia', 'Monitorowanie', 'Automatyzacja', and 'Pomoc'. The main content area displays the 'Przegląd' (Overview) tab for the virtual network. It shows the following details: Group zasobów (resource group): 'rg-projekt-chmurowy_asr'; Lokalizacja (location): 'Sweden Central'; Subskrypcja (subscription): 'Azure for Students'; Identyfikator subskrypcji (subscription ID): 'a8e6dffb-d6a3-448e-9254-ed7b992e75f0'; Przestrzeń adresowa (address space): '10.0.0.0/16'; Oglaszane prefiksy bramy (advertised gateway prefixes): '—'; Podsieci (subnets): 'Podsieć 1'; Serwery DNS (DNS servers): 'Usługa DNS dostarczana przez platformę Azure'; Ciąg społeczności BGP (BGP community string): 'Konfiguruj'; Identyfikator sieci wirtualnej (virtual network ID): 'acad980c-f480-43a2-a350-2e3090275aac'. At the bottom, there are tabs for 'Topologia' (Topology), 'Właściwości' (Properties), 'Możliwości' (Capabilities), 'Rekomendacje' (Recommendations), and 'Samouczki' (Tutorials).



Wdrażanie usług aplikacyjnych i bazodanowych

W tym etapie prac projektowych skoncentrowałem się na wdrożeniu właściwych usług aplikacyjnych oraz warstwy przechowywania danych, co stanowiło serce całego środowiska rozproszonego. Kontynuując edycję kodu w programie Visual Studio Code, uzupełniłem plik main.tf o definicje bezserwerowego środowiska kontenerowego oraz elastycznego serwera relacyjnej bazy danych. W pierwszej kolejności zadeklarowałem zasób `azurerm_container_app_environment` o nazwie `cae-projekt-chmurowy`. Ze względu na restrykcyjne limity subskrypcji studenckiej oraz niedostępność darmowych instancji w Europie Północnej, podjąłem celową decyzję o architekturze wieloregionowej (Multi-Region) i wymusiłem utworzenie tego środowiska w lokalizacji `germanywestcentral`. Na poziomie konfiguracji zintegrowałem je bezpośrednio z powołanym wcześniej systemem monitorowania, mapując parametr `log_analytics_workspace_id` do identyfikatora obszaru roboczego za pomocą dynamicznej referencji.

```
# 7. Środowisko uruchomieniowe dla aplikacji (Container Apps Environment)
resource "azurerm_container_app_environment" "cae" {
  name                = "cae-projekt-chmurowy"
  location            = "germanywestcentral"
  resource_group_name = azurerm_resource_group.rg.name

  # Łączymy środowisko z naszym monitoringiem, który stworzyliśmy wcześniej
  log_analytics_workspace_id = azurerm_log_analytics_workspace.law.id
}
```

```
C:\Users\adria\Terraform>terraform apply
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/a8e6dff6-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr]
azurerm_virtual_network.vnet: Refreshing state... [id=/subscriptions/a8e6dff6-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy]
azurerm_log_analytics_workspace.law: Refreshing state... [id=/subscriptions/a8e6dff6-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.OperationalInsights/workspaces/law-projekt-chmurowy]
azurerm_postgresql_flexible_server.db: Refreshing state... [id=/subscriptions/a8e6dff6-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.DBforPostgreSQL/flexibleServers/projekt-asr-179993]
azurerm_subnet.subnet: Refreshing state... [id=/subscriptions/a8e6dff6-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy/subnets/sn-et-aplikacja]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# azurerm_container_app_environment.cae will be created
+ resource "azurerm_container_app_environment" "cae" {
  + default_domain      = (known after apply)
  + docker_bridge_cidr  = (known after apply)
  + id                  = (known after apply)
  + infrastructure_resource_group_name = (known after apply)
  + internal_load_balancer_enabled = false
  + location            = "germanywestcentral"
  + log_analytics_workspace_id = "/subscriptions/a8e6dff6-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.OperationalInsights/workspaces/law-projekt-chmurowy"
  + name                = "cae-projekt-chmurowy"
  + platform_reserved_cidr = (known after apply)
  + platform_reserved_dns_ip_address = (known after apply)
  + resource_group_name = "rg-projekt-chmurowy_asr"
  + static_ip_address   = (known after apply)
  + zone_redundancy_enabled = false
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

azurerm_container_app_environment.cae: Creating...
azurerm_container_app_environment.cae: Still creating... [00m10s elapsed]
azurerm_container_app_environment.cae: Still creating... [00m20s elapsed]
azurerm_container_app_environment.cae: Still creating... [00m30s elapsed]
azurerm_container_app_environment.cae: Still creating... [00m40s elapsed]
azurerm_container_app_environment.cae: Still creating... [00m50s elapsed]
azurerm_container_app_environment.cae: Still creating... [01m00s elapsed]
azurerm_container_app_environment.cae: Creation complete after 1m8s [id=/subscriptions/a8e6dff6-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.App/managedEnvironments/cae-projekt-chmurowy]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```



Następnie przeszedłem do zaimplementowania warstwy bazodanowej poprzez deklarację zasobu `azurerm_postgresql_flexible_server` o unikalnej nazwie `projekt-asr-179993`. Serwer ten ulokowałem z powrotem w Szwecji (`swedencentral`), co pozwoliło mi na przetestowanie komunikacji między usługami rozproszonymi geograficznie w różnych regionach Azure. W kodzie zdefiniowałem parametry bezpieczeństwa, takie jak login i hasło administratora, wskazałem wersję silnika bazy danych oraz określiłem strefę dostępności. Mając na uwadze optymalizację kosztów w ramach konta laboratoryjnego, wybrałem najniższy dostępny profil wydajnościowy `SKU B_Standard_B1ms` i ograniczyłem przestrzeń dyskową do 32 GB.

```
# 8. Serwer bazy danych PostgreSQL
resource "azurerm_postgresql_flexible_server" "db" {
  name                        = "projekt-asr-179993"
  resource_group_name        = azurerm_resource_group.rg.name
  location                   = azurerm_resource_group.rg.location
  version                    = "14"
  administrator_login        = "dbadmin"
  administrator_password     = "BardzoTrudneHaslo123!"
  zone                       = "1"
  storage_mb                 = 32768
  sku_name                   = "B_Standard_B1ms"
}
```

Po sfinalizowaniu struktury kodu w Visual Studio Code, otworzyłem zintegrowany terminal i wywołałem polecenie `terraform plan`, aby sprawdzić, czy narzędzie poprawnie zinterpretowało nowo dodane zależności między bazą, aplikacją a siecią. Po pomyślnej weryfikacji i upewnieniu się, że konfiguracja nie zawiera błędów, uruchomiłem komendę `terraform apply`. Krok ten zainicjował proces wysyłania zapytań do API Azure i doprowadził do pełnej materializacji oraz uruchomienia stabilnych usług aplikacyjnych i bazodanowych w chmurze.

```
C:\Users\adria\Terraform>terraform apply
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy]
azurerm_virtual_network.vnet: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy]
azurerm_log_analytics_workspace.law: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy/providers/Microsoft.OperationalInsights/workspaces/law-projekt-chmurowy]
azurerm_subnet.subnet: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy/subnets/subnet-a-plikacja]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# azurerm_container_app_environment.cae will be created
+ resource "azurerm_container_app_environment" "cae" {
  + default_domain      = (known after apply)
  + docker_bridge_cidr  = (known after apply)
  + id                  = (known after apply)
  + infrastructure_resource_group_name = (known after apply)
  + internal_load_balancer_enabled = false
  + location             = "polandcentral"
  + log_analytics_workspace_id = "/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy/providers/Microsoft.OperationalInsights/workspaces/law-projekt-chmurowy"
  + name                = "cae-projekt-chmurowy"
  + platform_reserved_cidr = (known after apply)
  + platform_reserved_dns_ip_address = (known after apply)
  + resource_group_name  = "rg-projekt-chmurowy"
  + static_ip_address    = (known after apply)
  + zone_redundancy_enabled = false
}

# azurerm_postgresql_flexible_server.db will be created
+ resource "azurerm_postgresql_flexible_server" "db" {
  + administrator_login      = "dbadmin"
  + administrator_password   = (sensitive value)
  + auto_grow_enabled        = false
  + backup_retention_days    = (known after apply)
  + fqdn                    = (known after apply)
  + geo_redundant_backup_enabled = false
  + storage_tier              = (known after apply)
  + version                  = "14"
  + zone                    = "1"
  + authentication (known after apply)
}
```

Próba zautomatyzowania zarządzania użytkownikami (Microsoft Entra ID)

W tej części podjąłem próbę rozbudowy infrastruktury o zaawansowane mechanizmy zarządzania tożsamością i uprawnieniami (RBAC). Powróciłem do edycji pliku main.tf w Visual Studio Code, dodając do konfiguracji nowego dostawcę usług (providera) azuread, który odpowiada za komunikację z usługą Microsoft Entra ID (dawniej Azure Active Directory).

```
# 9. Dodawanie użytkowników do usługi Microsoft Entra ID
resource "azuread_user" "developer" {
  user_principal_name = "programista179993@${data.azuread_domains.default.domains[0].domain_name}"
  display_name       = "Jan Programista (179993)"
  password           = "ZaszyfrowaneHasloDev123!"
  force_password_change = false
}

resource "azuread_user" "auditor" {
  user_principal_name = "audytor179993@${data.azuread_domains.default.domains[0].domain_name}"
  display_name       = "Anna Audytor (179993)"
  password           = "ZaszyfrowaneHasloAudit123!"
  force_password_change = false
}
```

Następnie zadeklarowałem w kodzie zasoby mające na celu zautomatyzowane utworzenie kont dwóch nowych użytkowników oraz przypisanie im zróżnicowanych ról dostępowych (współtwórcy oraz czytelnika) w obrębie mojej grupy zasobów.

```
# 10. Przypisywanie ról (RBAC) - Przenoszenie użytkowników do odpowiednich poziomów dostępu
# Programista otrzymuje rolę Contributor (Współtwórca) - może zarządzać infrastrukturą w grupie
resource "azurerms_role_assignment" "dev_access" {
  scope           = azurerms_resource_group.rg.id
  role_definition_name = "Contributor"
  principal_id    = azuread_user.developer.object_id
}

# Audytor otrzymuje rolę Reader (Czytelnik) - może jedynie przeglądać zasoby i czytać logi
resource "azurerms_role_assignment" "auditor_access" {
  scope           = azurerms_resource_group.rg.id
  role_definition_name = "Reader"
  principal_id    = azuread_user.auditor.object_id
}
```

Po zapisaniu zmian w pliku, uruchomiłem w zintegrowanym terminalu komendę terraform init, aby pobrać wymagane wtyczki nowego dostawcy, a po ich zainstalowaniu zainicjowałem proces wdrożenia poleceniem terraform apply. W trakcie wykonywania kodu napotkałem jednak na krytyczny błąd autoryzacji oznaczony komunikatem 403 Forbidden: Insufficient privileges:

```
C:\Users\adria\Terraform>terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Finding latest version of hashicorp/azurerm...
- Using previously-installed hashicorp/azurerm v3.100.0
- Installing hashicorp/azurerm v3.8.0...
- Installed hashicorp/azurerm v3.8.0 (signed by HashiCorp)
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

C:\Users\adria\Terraform>terraform apply
data.azuread_domains.default: Read complete after 1s [id=domainsd81a2e37-d18e-42ec-8839-9a2de1586e0cfzpq0K0Lnlco7Q1Bwt-xve0MDE8=]
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr]
azurerm_virtual_network.vnet: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy]
azurerm_log_analytics_workspace.law: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.OperationalInsights/workspaces/law-projekt-chmurowy]
azurerm_postgresql_flexible_server.db: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.DBforPostgreSQL/flexibleServers/projekt-asr-179993]
azurerm_subnet.subnet: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.Network/virtualNetworks/vnet-projekt-chmurowy/subnets/snet-aplikacja]
azurerm_container_app_environment.cae: Refreshing state... [id=/subscriptions/a8e6dffb-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.App/managedEnvironments/cae-projekt-chmurowy]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Error: Creating user "programista179993@o365.prz.edu.pl"

    with azurerm_user.developer,
    on main.tf line 80, in resource "azurerm_user" "developer":
    80: resource "azurerm_user" "developer" {

unexpected status 403 (403 Forbidden) with error: Authorization_RequestDenied: Insufficient privileges to complete the operation.

Error: Creating user "audytor179993@o365.prz.edu.pl"

    with azurerm_user.auditor,
    on main.tf line 87, in resource "azurerm_user" "auditor":
    87: resource "azurerm_user" "auditor" {

unexpected status 403 (403 Forbidden) with error: Authorization_RequestDenied: Insufficient privileges to complete the operation.
```

Szczegółowa analiza logów w terminalu pozwoliła mi ustalić, że środowisko edukacyjne, na którym operuję, jest na stałe zintegrowane z głównym katalogiem (tenantem) uczelni (@o365.prz.edu.pl). Z powodu globalnych polityk bezpieczeństwa narzuconych przez administratorów IT, moje konto studenckie nie posiadało uprawnień administracyjnych (Global Administrator) wymaganych do tworzenia nowych obiektów w uczelnianej bazie tożsamości.

Zinterpretowałem ten błąd jako twarde, zewnętrzne ograniczenie narzucone na środowisko edukacyjne. Aby utrzymać poprawny i wolny od błędów stan infrastruktury, wycofałem się z tej modyfikacji, usuwając dodany blok zarządzania tożsamością z pliku main.tf. Sytuacja ta pozwoliła mi jednak w praktyce zapoznać się z restrykcjami i mechanizmami IAM (Identity and Access Management), które są kluczowym elementem bezpieczeństwa w realnych, produkcyjnych i korporacyjnych środowiskach chmurowych.



Wdrożenie aplikacji kontenerowej i konfiguracja dostępu sieciowego

W dalszej części prac projektowych skoncentrowałem się na uruchomieniu docelowej aplikacji, ożywiając tym samym zarysowaną wcześniej infrastrukturę. Prace w edytorze Visual Studio Code rozpocząłem od zalecanej konfiguracji związanej z nieudaną próbą zarządzania tożsamością, dostosowując kod do ograniczeń subskrypcji studenckiej. Następnie na końcu pliku zadeklarowałem nowy zasób `azurerm_container_app` o nazwie `app-projekt-179993`, który powiązałem logicznie z moją grupą zasobów oraz wdrożonym w trzecim etapie środowiskiem kontenerowym (Container App Environment).

W sekcji konfiguracyjnej template precyzyjnie określiłem zasoby obliczeniowe dla aplikacji, przydzielając jej 0,25 rdzenia CPU oraz 0,5 GB pamięci RAM, co w pełni odpowiadało na zapotrzebowanie prostych usług internetowych. Na potrzeby weryfikacji poprawności wdrożenia i łączności, zastosowałem oficjalny, testowy obraz kontenera dostarczany przez firmę Microsoft (`hello-world`). Kluczowym elementem tego etapu było odpowiednie skonfigurowanie bloku `ingress`. Zezwoliłem na ruch zewnętrzny (włączając flagę `external_enabled`), zablokowałem połączenia nieszyfrowane i przekierowałem 100% ruchu sieciowego na port 80 nowej rewizji aplikacji.

```
# 9. Wdrożenie aplikacji kontenerowej (Container App)
resource "azurerm_container_app" "app" {
  name                       = "app-projekt-179993"
  container_app_environment_id = azurerm_container_app_environment.cae.id
  resource_group_name       = azurerm_resource_group.rg.name
  revision_mode              = "Single"

  template {
    container {
      name     = "hello-world-app"
      image    = "mcr.microsoft.com/azuredocs/containerapps-helloworld:latest"
      cpu      = 0.25
      memory   = "0.5Gi"
    }
  }

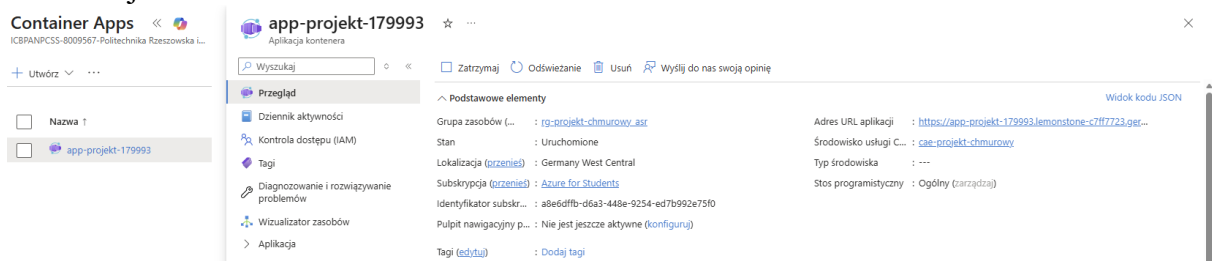
  ingress {
    allow_insecure_connections = false
    external_enabled           = true
    target_port                 = 80

    traffic_weight {
      percentage      = 100
      latest_revision = true
    }
  }
}
```

Po weryfikacji i zapisaniu zmian, otworzyłem zintegrowany terminal w VS Code. W pierwszej kolejności wykonałem polecenie `terraform init`, aby odświeżyć stan i zaktualizować moduły do obecnego kształtu kodu. Następnie uruchomiłem polecenie `terraform apply`. Po przeanalizowaniu przedstawionego planu wdrożenia (dodanie jednego zasobu) i zatwierdzeniu `go`, Terraform z powodzeniem skomunikował się z interfejsem API chmury Azure, tworząc aplikację i eksponując ją w internecie pod wygenerowanym, publicznym adresem URL.

```
azurew_container_app-app: Creating...  
azurew_container_app-app: Still creating... [60m10s elapsed]  
azurew_container_app-app: Still creating... [90m20s elapsed]  
azurew_container_app-app: Creation complete after 23s [id-/subscriptions/a8e6dff8-d6a3-448e-9254-ed7b992e75f0/resourceGroups/rg-projekt-chmurowy_asr/providers/Microsoft.App/containerApps/app-projekt-179993]  
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Widok panelu usługi Azure Container Apps potwierdzający poprawne wdrożenie aplikacji docelowej:



Zrzut ekranu udowadnia zakończenie projektu sukcesem – widoczny jest aktywny stan działania kontenera ("Uruchomione"), poprawne przypisanie do środowiska "cae-projekt-chmurowy" oraz wygenerowany publiczny adres URL umożliwiający dostęp do aplikacji z poziomu przeglądarki.

Na końcu dodałem zabezpieczenie kodu źródłowego, odizolowanie plików stanu i sekretów za pomocą mechanizmu .gitignore w folderze projektowym, skrócony opis projektowy README oraz opublikowanie kompletnego repozytorium wraz z dokumentacją w serwisie GitHub.



Podsumowanie

W tym projekcie, udało się z sukcesem zaprojektować, zautomatyzować i wdrożyć w pełni funkcjonalną, rozproszoną architekturę chmurową w środowisku Microsoft Azure przy użyciu narzędzia Terraform. Zbudowana infrastruktura obejmuje solidne fundamenty w postaci dedykowanej sieci wirtualnej z podziałem na podsieci, scentralizowanego systemu monitorowania opartego na Log Analytics Workspace oraz warstwy bazodanowej wykorzystującej elastyczny serwer PostgreSQL. Zwieńczeniem prac było pomyślne wdrożenie docelowej aplikacji kontenerowej w usłudze Azure Container Apps, która została poprawnie wyeksponowana na ruch publiczny w internecie. Osiągnięcie tego stanu stanowiło ostateczny dowód na to, że cały napisany kod jest spójny i działa prawidłowo w praktyce.

Realizacja projektu wiązała się z pewnymi wyzwaniami technicznymi, które wymagały analizy logów i zrozumienia specyfiki platformy Azure. Pierwszą z przeszkód był brak domyślnej rejestracji niezbędnego dostawcy usług kontenerowych na subskrypcji studenckiej. Największym problemem, z jakim przyszło się zmierzyć, była jednak próba implementacji zaawansowanych mechanizmów zarządzania tożsamością (IAM) za pomocą dostawcy Microsoft Entra ID. Ze względu na rygorystyczne polityki bezpieczeństwa i ograniczenia narzucone przez głównego administratora w uczelnianym katalogu (tenant), Terraform napotkał krytyczny błąd braku uprawnień podczas prób tworzenia nowych użytkowników.

Pomyślne ukończenie tego zadania doskonale obrazuje, jak ogromną wartość w nowoczesnym IT niesie ze sobą podejście Infrastructure as Code, projektowanie środowisk w ten sposób niemal całkowicie eliminuje konieczność ręcznego tworzenia zasobów przez portal graficzny, co drastycznie zmniejsza ryzyko błędów. Największą zaletą takiego rozwiązania jest powtarzalność – raz napisaną infrastrukturę można wdrożyć w kilka minut na dowolnym innym koncie lub w innym regionie geograficznym. Ponadto trzymanie definicji serwerów i sieci w postaci zwykłego kodu tekstowego pozwala na stosowanie systemów kontroli wersji, takich jak Git. Dzięki temu cała architektura chmurowa staje się naturalnie udokumentowana, łatwa w audytowaniu i w pełni gotowa do pracy w profesjonalnych potokach CI/CD.